



UNIVERSIDADE FEDERAL DO CEARÁ
Campus de Quixadá
Prof. Antônio Luiz
QXD01113 - Estrutura de Dados Avançada - Turma 01A

AP1
2024.1

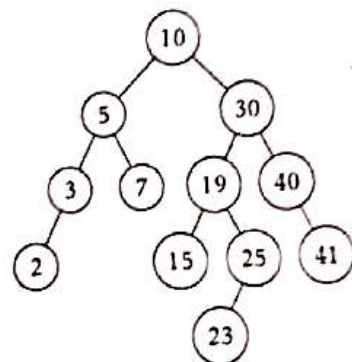
Nome: _____

ATENÇÃO: Coloque o seu nome completo em todas as folhas que forem entregues. Folhas sem identificação não serão corrigidas. A prova pode ser feita com caneta azul ou preta, ou também a lápis contanto que fique legível. Não é permitido usar calculadora nem nenhum outro dispositivo.

- [2 pontos] 1. Seja T uma árvore AVL inicialmente vazia. Mostre (desenhe) cada uma das árvores AVL intermediárias que resultam após a inserção bem-sucedida das chaves ~~30~~, ~~40~~, ~~15~~, ~~25~~, ~~10~~, ~~20~~, ~~5~~, ~~3~~, ~~7~~, ~~2~~, ~~41~~ (nesta ordem) na árvore T . Ao fazer o seu desenho, para cada nó da árvore, escreva o balanço do nó ao lado do nó. Para cada vez que uma operação de rotação acontecer, escreva qual tipo de rotação está sendo executada (existem 4 tipos) e qual o nó pivô da rotação.

Atenção: siga o algoritmo que foi ensinado. Para cada rotação errada será decrementado 0.25 da nota.

- [2 pontos] 2. Considere o seguinte enunciado:
Dada a árvore AVL da figura ao lado, desenhe cada uma das árvores AVL intermediárias que resultam da remoção das chaves ~~10~~, ~~30~~, ~~15~~, ~~25~~, ~~2~~, ~~3~~ (nesta ordem). Ao fazer o seu desenho, para cada nó da árvore, escreva o balanço do nó ao lado do nó. Para cada vez que uma operação de rotação acontecer, escreva qual rotação está sendo executada e qual o nó pivô da rotação.
Atenção 1: siga o algoritmo que foi ensinado em sala. Para cada rotação errada será decrementado 0.25 da nota.
Atenção 2: ao remover uma chave, pode ser que mais de uma rotação venha a ser necessária até que a árvore volte a ser AVL. Faça todas as rotações necessárias antes de remover a próxima chave. Se você não fizer isso, sua solução estará errada.



- [2 pontos] 3. Escreva em C++ uma função chamada `constroi_Max_Heap(int A[], int n)`, que recebe como entrada um vetor A com n inteiros e o tamanho n , e rearranja os elementos de A de modo que ele torne-se um heap binário de máximo. A sua função deve ter complexidade de pior caso $O(n)$. Ademais, todo o código necessário para executar a sua função deve estar escrito na sua solução. Se for usada alguma função cujo código não for fornecido junto com a solução, então será descontado 1 ponto por cada função usada. Após escrever o algoritmo, execute-o usando como entrada o vetor

$A = [5, 3, 17, 10, 84, 19, 6, 22, 9]$.

Você deve mostrar/desenhar o vetor resultante da execução do seu algoritmo.

- [2 pontos] 4. Demonstre o que acontece quando inserimos as chaves 12, 44, 13, 88, 23, 94, 11, 39, 20, 16 e 5 em uma tabela de espalhamento (tabela hash) com colisões resolvidas por encadeamento (listas encadeadas). Considere uma tabela com onze posições e a função de espalhamento $h(k) = (2k + 5) \bmod 11$.
- [1 ponto] 5. Considere a inserção das chaves 2, 32, 43, 16, 77, 51, 1, 17, 42 em uma tabela hash de comprimento $M = 17$ usando endereçamento aberto com a função hash $h(k) = k \bmod 17$. Ilustre o resultado da inserção dessas chaves utilizando sondagem linear. Para cada chave, indique na sua solução todas as tentativas de inserção que foram feitas.

Nota: _____

6. Uma **fila de prioridades** é uma estrutura de dados que mantém uma coleção A de elementos na qual cada um de seus elementos possui uma *prioridade*, isto é, um valor numérico. As filas de prioridades existem em duas formas: **filas de prioridades mínimas** e **filas de prioridades máximas**. Em sala de aula, vimos como implementar de forma teórica uma fila de prioridade máxima, de forma eficiente, usando como base uma outra estrutura chamada *heap de máximo*. Nesta questão, a estrutura de dados **MaxPriorityQueue** é uma fila de prioridade de máximo, onde os seus elementos são números inteiros (as prioridades). Essa estrutura de dados possui basicamente as 7 funções públicas listadas a seguir:

- o construtor **MaxPriorityQueue()**, que cria uma fila de prioridades vazia em tempo $O(1)$.
- o construtor **MaxPriorityQueue(int vec[], int n)**, que recebe como parâmetro o vetor **vec** e seu tamanho n e cria uma fila de prioridades em tempo $O(n)$, contendo todos os elementos do vetor **vec**.
- a função **top()** que retorna o valor do elemento de maior prioridade (mas não o remove da fila). Essa função tem complexidade $O(1)$.
- a função **pop()** que retorna o valor do elemento de maior prioridade e o remove da fila. Essa função executa em tempo $O(\lg n)$.
- a função **push(int v)** que recebe um inteiro v e o insere na fila em tempo $O(\lg n)$. Essa função não retorna nada.
- a função **empty()** que retorna o valor booleano **true** se, e somente se, a fila estiver vazia. Essa função tem complexidade $O(1)$.
- a função **size()** que retorna um valor inteiro, que é o número de elementos atualmente na fila. Essa função tem complexidade $O(1)$.

Com base nos conhecimentos adquiridos com filas de prioridades, resolva o problema a seguir.

- (a) [1 ponto] Usando a estrutura de dados **MaxPriorityQueue** e suas funções públicas definidas acima, escreva em C++ um algoritmo que ordena um vetor de inteiros A com n inteiros, em tempo $O(n \lg n)$. Você deve escrever uma função chamada **ordena_vetor(int A[], int n)** que recebe como entrada o vetor A com n inteiros e o seu tamanho n e o ordena. O retorno dessa função é do tipo **void**. Dentro dessa função, ela deve fazer uso da estrutura de dados **MaxPriorityQueue** definida acima. Ao final, prove porque a complexidade da sua função é $O(n \lg n)$.